LOW-LEVEL DESIGN

for

Cloud Forensics Automation for Retail Breaches

SAIBIN K SIJU
Register No: 23BCCED028
Course: Bachelor of Computer Applications (BCA)
Yenepoya Institute of Arts, Science, Commerce and Management

Date: 21/04/2026

Under the guidance of

Ms. Prathiksha
Lecturer, Department of Computer Science
Yenepoya Institute of Arts, Science, Commerce and Management

Submitted to

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT
Balmatta, Mangalore
Yenepoya (Deemed to be University)

TABLE OF CONTENTS

Section No.	Title	Page No.
1	Introduction	3
1.1	Purpose of Document	3
1.2	Scope of the System	3
1.3	Definitions & Abbreviations	4
2	System Overview	5
2.1	System Description	5
2.2	Architecture Overview	5
2.3	Technology Stack	5
3	Module Design	6
4	Class Design & Object Interaction	7
4.1	Class Diagram	7
4.2	Class Descriptions	8
4.3	Object Initialization Flow	9
5	Sequence Diagrams	9-10
6	API Specifications	11
7	Data Design	12
8	Security Considerations	13
9	Error Handling & Edge Cases	14
10	Logging System	15
11	Deployment Considerations	16
12	Testing Strategy	16
13	Future Enhancements	17
14	Conclusion	18

1. Introduction

1.1 Purpose of Document

This document provides a detailed Low-Level Design (LLD) for the Cloud Forensics Automation for Retail Breaches project (PRJN26-172). It describes the internal architecture, module-level design, class interactions, sequence flows, and component responsibilities required to implement the forensic file evidence collection system.

The purpose of this document is to:

- Define detailed system components, classes, and their methods
- Describe file scanning, SHA256 hash comparison, and tamper detection flows
- Provide technical design for developers and evaluators
- Document API specifications with request/response formats and status codes
- Define security controls, logging design, and testing strategy

1.2 Scope of the System

The Cloud Forensics Automation for Retail Breaches system is designed to monitor files in a retail environment's digital directories and detect unauthorized modifications. It simulates a real-world retail data breach investigation scenario.

It provides:

- Automated file scanning of a monitored directory (scan_folder)
- SHA256 cryptographic hash computation for tamper detection
- File status classification: NEW, UNCHANGED, or MODIFIED
- Forensic evidence stored in PostgreSQL with automated risk level classification (CRITICAL/HIGH/MEDIUM/LOW)
- Flask-based admin dashboard with REST API
- Client interface for file upload, view, and edit operations
- Real-time automatic scan via watchdog Observer (watcher.py) on any file system change

1.3 Definitions & Abbreviations

This section defines key terms used throughout the document.

Definitions

- **SHA256 Hash:** A 256-bit cryptographic digest of a file's content. Any byte-level change to the file produces a completely different hash, making it ideal for tamper detection.
- **Forensic Evidence:** A permanent record of file metadata, hash values, and status stored in PostgreSQL and captured at each scan session.
- **Scan Session:** A single execution of the scan_directory() function that inspects all files in scan_folder and records their current state.
- **Tamper Detection:** The process of comparing a file's current SHA256 hash against its previously recorded hash to determine if the file was modified.
- **RBAC:** Role-Based Access Control — restricting system access based on the role of the authenticated user.

Abbreviations

Abbreviation	Full Form
SHA256	Secure Hash Algorithm 256-bit
CSV	Comma-Separated Values
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
CSRF	Cross-Site Request Forgery
LLD	Low-Level Design
HLD	High-Level Design
WSGI	Web Server Gateway Interface

2. System Overview

2.1 System Description

The system is implemented as a Python Flask web application with a two-layer architecture: a forensic backend engine and a web interface. The forensic engine scans a monitored directory, computes SHA256 hashes of all files, compares them against previously recorded hashes, and classifies each file as NEW, UNCHANGED, MODIFIED, or DELETED. Results are inserted into PostgreSQL. The Flask web interface provides an admin dashboard for monitoring evidence and a client interface for file management.

2.2 Architecture Overview

The system follows a two-tier architecture separating the forensic logic from the web interface.

- Presentation Layer (Frontend): HTML5 + CSS3 + Vanilla JavaScript dashboard rendered by Flask's Jinja2 template engine. Provides the admin forensics dashboard and the client file management interface.
- Application Layer (Backend): Python Flask handles all routing, business logic, REST API endpoints, and file operations. The forensic engine (forensic_collector.py) is responsible for all scanning and hash computation.
- Data Layer: PostgreSQL (forensics_db) serves as the primary evidence store. All scan results are inserted via db.py. CSV is generated on-demand only via /api/export for reporting purposes. The system follows a hybrid approach where PostgreSQL ensures reliable and queryable storage, while CSV provides portable and human-readable reporting for forensic analysts.

2.3 Technology Stack

Layer	Technology	Purpose
Backend	Python 3.8+ / Flask 2.x	Web framework, routing, API endpoints
Forensic Engine	hashlib (SHA256)	Cryptographic file tamper detection
File Metadata	os module	File creation and modification timestamps
Evidence Storage	PostgreSQL (psycopg2)	Primary evidence storage — all scan results persisted here
Excel Export	watchdog (Observer)	Real-time file system event monitoring
Frontend	HTML5 + CSS3 + JS	Dashboard UI, file management

Layer	Technology	Purpose
		interface
Template Engine	Jinja2 (via Flask)	Server-side HTML rendering
Security	Werkzeug secure_filename()	File upload path traversal prevention

3. Module Design

The system is divided into four core implementation modules, each with a clearly defined responsibility.

Module	Name	Key Operations	Output
MODULE 1	File Ingestion	Sanitize filename. Save to scan_folder. Trigger scan.	Saved file + scan event
MODULE 2	Forensic Scan Engine	Walk scan_folder. Compute SHA256. Compare hashes. Assign status.	Evidence rows list
MODULE 3	Evidence Storage	Insert scan rows into PostgreSQL evidence table via db.py. Supports NEW, MODIFIED, UNCHANGED, DELETED statuses.	PostgreSQL evidence table
MODULE 4	Dashboard & API	Query PostgreSQL for latest scan. Serve JSON to admin dashboard. Generate CSV on-demand for export.	JSON API responses (PostgreSQL queries)

Module Descriptions

1. ForensicEngine Module

Handles all hashing and scanning logic. Walks the scan_folder directory, computes SHA256 hashes, loads previous hashes from PostgreSQL via db.py, and assigns a status (NEW, UNCHANGED, MODIFIED, or DELETED) to each file.

2. ReportManager Module

Manages all PostgreSQL database operations. Handles INSERT of scan results, SELECT queries for the dashboard API, summary statistics via GROUP BY, and scan history aggregation. CSV is only produced on demand via /api/export — never used as storage.

3. FileHandler Module

Manages all file upload and editing operations. On upload, saves file to scan_folder and triggers scanner.py via subprocess. On save via /save/<filename>, computes new SHA256 and updates PostgreSQL evidence table directly with MODIFIED status. Validates whether a file is text-readable before exposing edit functionality.

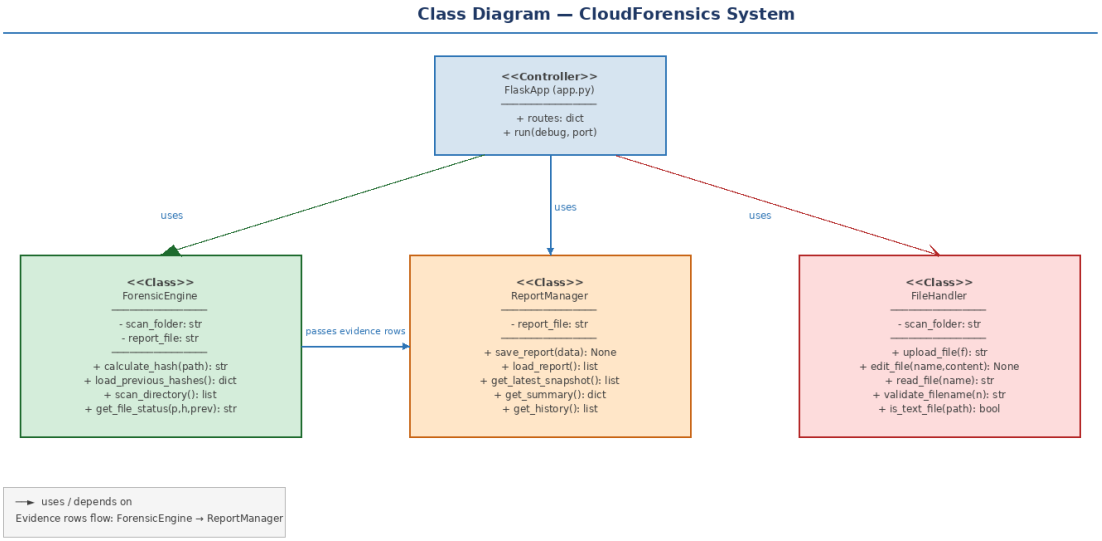
4. Flask API Layer

Acts as the controller. Handles all HTTP routing, template rendering, JSON responses, and triggers forensic scans. Connects to PostgreSQL via psycopg2 for all evidence queries. Manages /api/export for CSV download and /api/history for scan trend data.

4. Class Design & Object Interaction

4.1 Class Diagram

The diagram below shows the three core classes of the system, their attributes, methods, and relationships.



4.2 Class Descriptions

ForensicEngine

Handles all hashing and scanning logic.

- Attributes: scan_folder (str), report_file (str)
- calculate_hash(file_path: str) -> str: Reads file in 4096-byte chunks and returns SHA256 hexdigest.
- load_previous_hashes() -> dict: Calls db.get_previous_hashes() — queries PostgreSQL for the previous scan's hashes per file path.
- scan_directory() -> list: Walks scan_folder, computes hashes, assigns status (NEW/UNCHANGED/MODIFIED/DELETED). Detects deleted files by comparing previous paths against current filesystem.
- get_file_status(path, hash, prev_hashes) -> str: Returns NEW, UNCHANGED, MODIFIED, or DELETED. Also calls risk_level(status) to assign CRITICAL/HIGH/MEDIUM/LOW.

ReportManager

Manages all PostgreSQL evidence operations (INSERT, SELECT, GROUP BY aggregation).

- Attributes: report_file (str)
- insert_data(rows: list) -> None: Inserts scan rows into PostgreSQL evidence table. Primary evidence storage method — no CSV file is written during normal scan operation.
- get_current_state() -> list[dict]: Queries evidence table for the latest scan_time snapshot.
- get_latest_snapshot(rows: list) -> list: Returns only the most recent row per file path.
- get_summary() -> dict: SQL GROUP BY status query for total, modified, new, unchanged, deleted counts.
- get_history() -> list: SQL GROUP BY scan_time query returning modified/new/unchanged counts per session.

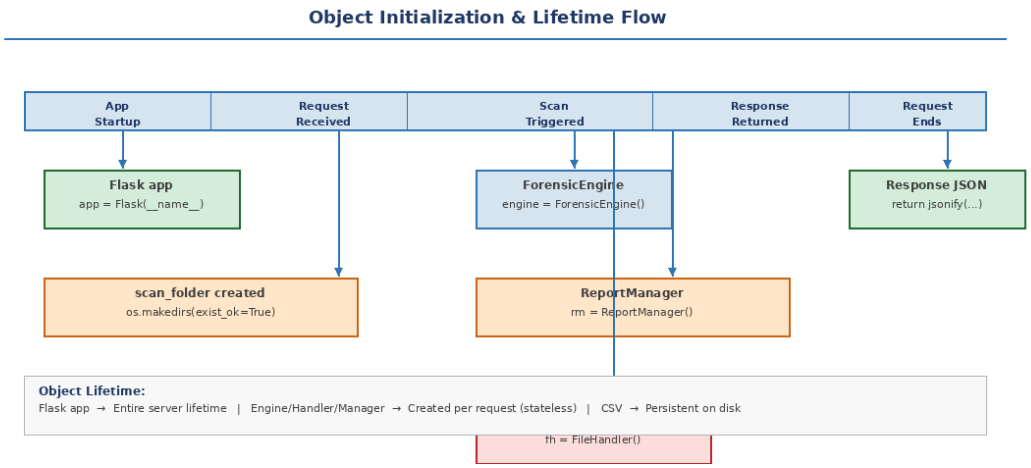
FileHandler

Manages file upload, editing, and validation.

- Attributes: scan_folder (str)
- upload_file(file: FileStorage) -> str: Validates, sanitizes, and saves uploaded file.
- edit_file(filename: str, content: str) -> None: Writes new content to existing file.
- validate_filename(filename: str) -> str: Applies secure_filename() sanitization.
- is_text_file(filepath: str) -> bool: Checks if file is UTF-8 readable.

4.3 Object Initialization Flow

The diagram below shows when each object is created and its lifetime within the system.

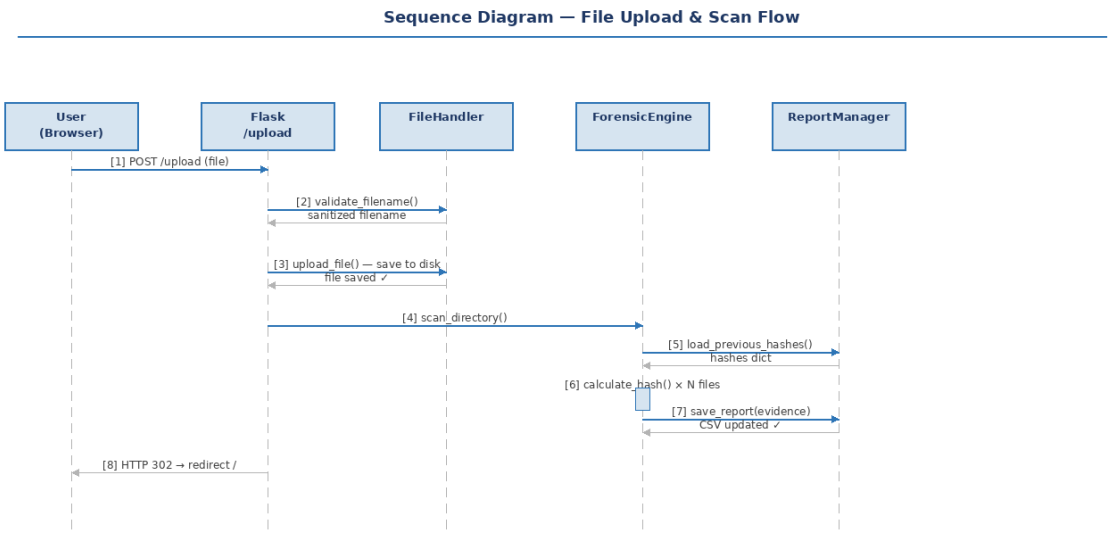


Object	Created When	Lifetime
Flask app	app.py executed	Entire server process lifetime (app.py) / watcher process (watcher.py runs separately)
ForensicEngine	Per scan request	Single request — stateless
ReportManager	Per API request	Single request — stateless
FileHandler	Per upload/edit request	Single request — destroyed after response
PostgreSQL evidence table	CREATE TABLE evidence executed in PostgreSQL	Persistent in PostgreSQL. Queryable, supports transactions, indexed on scan_time

5. Sequence Diagrams

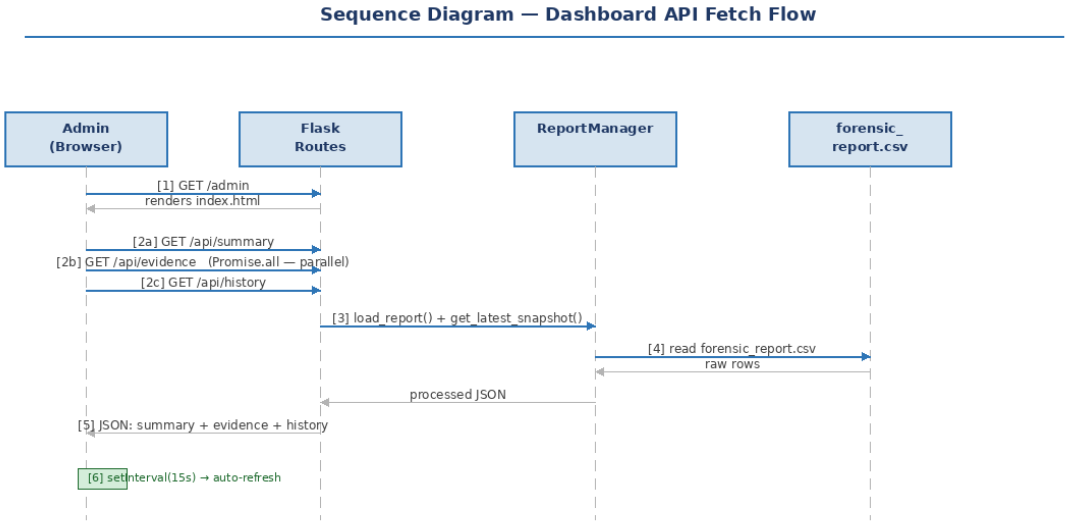
5.1 File Upload & Scan Flow

The diagram below shows the step-by-step interaction between system components when a file is uploaded and a scan is triggered.



5.2 Dashboard API Fetch Flow

The diagram below shows how the admin dashboard fetches forensic data from the backend APIs in parallel.



Note: All three API calls (/api/summary, /api/evidence, /api/history) are triggered in parallel using JavaScript Promise.all() to minimize dashboard load time.

6. API Specifications

All API endpoints are served by the Flask backend. Responses are JSON. HTTP status codes follow REST conventions.

6.1 GET /api/summary

Returns dashboard stat cards including file counts and risk assessment.

```
Response (200):
{ "total": 3, "modified": 1, "new": 0, "unchanged": 2,
  "risk_level": "HIGH", "deleted": 0, "last_scan": "2026-04-21
10:54:58" }
Status Codes: 200 — Success | 500 — Internal error
```

6.2 GET /api/evidence

Returns the latest snapshot per file, sorted MODIFIED first, then NEW, then UNCHANGED.

```
Response (200):
[ { "file_name": "sales_data.txt", "sha256": "4f9c00...",
    "status": "MODIFIED", "risk_level": "HIGH", "scan_time": "2026-04-
21 10:54:58" } ]
Status Codes: 200 — Success | 500 — Failed to load report
```

6.3 POST /api/scan

Triggers scanner.py via subprocess to run a fresh directory scan. scanner.py computes SHA256 hashes, detects NEW/MODIFIED/DELETED files, and inserts results into PostgreSQL. Returns { "status": "ok" }.

```
Request Body: None
Response (200): { "success": true, "output": "Scan Completed." }
Response (409): { "success": false, "error": "Scan already in progress"
}
Status Codes: 200 — Success | 409 — Concurrent lock | 500 — Subprocess
error
```

6.4 POST /upload

Accepts a file upload from the client interface, saves it to scan_folder, and triggers a forensic scan.

```
Request: multipart/form-data — field name: file
Response (302): Redirect to / on success
Response (400): No file selected
Response (413): File exceeds 16 MB limit
```

6.5 POST /save/<filename>

Saves edited file content back to disk and triggers a forensic scan to log the change as MODIFIED.

```
Request: form-data — field: content (edited text)
Response (302): Redirect to /view/<filename>
Status Codes: 302 — Redirect | 404 — File not found
```

6.6 GET /api/download

Generates a CSV report dynamically from PostgreSQL evidence data. Queries the latest scan results, builds the CSV in memory using csv.writer, and returns it as a downloadable attachment. No CSV file is stored on disk.

```
Response: Content-Type: text/csv — forensic_evidence.csv generated
dynamically from PostgreSQL
Status Codes: 200 — Success | 404 — Report not found
```

7. Data Design & Storage Architecture

7.1 PostgreSQL Evidence Table Schema

The system uses PostgreSQL as the primary evidence storage. All scan results are inserted into the evidence table in real time. CSV is generated dynamically only for export and reporting purposes — no CSV file is maintained on disk as a data store.

Column Name	Data Type	Example	Description
File Name	String	sales_data.txt	Base filename — display only
File Path	String	scan_folder\file.txt	Unique key for deduplication
Created Time	Datetime	2026-03-04 13:53	os.getctime() — NOT reliable on Linux
Last Modified Time	Datetime	2026-04-06 10:54	os.getmtime() — primary tamper indicator

Column Name	Data Type	Example	Description
SHA256 Hash	String (64 hex)	4f9c00caf2...	Cryptographic file fingerprint
Status	Enum	MODIFIED	NEW | UNCHANGED | MODIFIED
Scan Time	Datetime	2026-04-06 10:54:58	Groups rows into scan sessions

Note: os.getctime() is not reliable for forensic timestamp on Linux — it returns inode change time, not file creation time. os.getmtime() (last modified) is the primary tamper detection field.

7.2 PostgreSQL Evidence Table

The system now uses PostgreSQL as the evidence store. The CREATE TABLE statement for the evidence table is:

```
CREATE TABLE evidence (
    id            SERIAL PRIMARY KEY,
    file_name     TEXT,
    file_path     TEXT,
    sha256        TEXT,
    status        TEXT,
    risk_level    TEXT,
    created_time  TIMESTAMP,
    modified_time TIMESTAMP,
    scan_time     TIMESTAMP
);
```

8. Security Considerations

8.1 Input Validation & Path Traversal Prevention

All uploaded and accessed filenames are sanitized using Werkzeug's secure_filename() before any disk operation. This strips path separators (../), null bytes, and special characters. For example, the filename ../../etc/passwd becomes etc_passwd after sanitization, preventing escape from scan_folder. File upload size is capped at 16 MB via MAX_CONTENT_LENGTH.

8.2 Subprocess Security

The /api/scan route calls subprocess.run(['python', 'forensic_collector.py']) with a fixed argument list — no user-provided input is ever passed as a subprocess argument. This prevents shell injection. The call does not use shell=True. The planned future refactor replaces the subprocess call with a direct engine.scan_directory() method call, eliminating the subprocess entirely.

8.3 Forensic Evidence Protection

PostgreSQL is the primary evidence store. Evidence rows are INSERT-only — no DELETE or UPDATE on historical records. Database credentials are in DB_CONFIG in app.py. Future: move credentials to .env environment variables and add database audit logging.

8.4 Planned Security Enhancements

Gap	Risk Level	Planned Fix
No authentication on /admin	HIGH	Flask-Login with session tokens
No CSRF protection	MEDIUM	Flask-WTF CSRFProtect on all POST routes
No rate limiting	MEDIUM	Flask-Limiter (10 scans/min limit)
DB credentials hardcoded in app.py	HIGH	Move credentials to environment variables (.env)
Subprocess call	LOW	Refactor to direct class method call

9. Error Handling & Edge Cases

9.1 Standard Error Scenarios

Scenario	Trigger	Handling Strategy
Hash calculation fails	PermissionError / OSError	Returns 'Error' string; scan continues. Logs to error.log.
scan_folder missing	Startup check	os.makedirs(exist_ok=True) auto-creates.
Database connection fails	psycopg2.OperationalError : wrong password, PostgreSQL not running	psycopg2.OperationalError raised. App returns 500. Fix: check DB_CONFIG password and ensure PostgreSQL is running.

Scenario	Trigger	Handling Strategy
No file in upload	POST /upload empty field	HTTP 400 — No file selected.
File not found	View / Edit / Download	HTTP 404 — File not found.
Subprocess scan fails	Python not found	API returns { "status": "error" }. scanner.py prints error to console. Future: return detailed error message in JSON response.
Concurrent scan request	Two simultaneous POSTs	Single-thread Flask serializes. Future: threading.Lock() — 409.

9.2 Edge Cases

- **Very Large Files (>1 GB):** The chunked 4096-byte read handles any file size without memory issues. Scan time increases linearly. Future: configurable max_file_size_to_hash setting.
- **Database query failure:** Database query failure: Invalid queries or corrupted data may raise psycopg2 errors. Future: per-row try/except in insert_data() — skip bad rows and log them.
- **Disk Full During Scan:** CSV export may fail if disk space is insufficient during file generation. Future: check available disk space before writing; log CRITICAL if below 10 MB threshold.
- **Empty scan_folder:** scan_directory() returns an empty list. Dashboard shows 0 files with LOW risk — correct forensic behavior.

10. Logging System

The current version prints to terminal only. The following logging design is required for forensic traceability — all events must be permanently recorded.

Log Level	Log File	Events Logged
INFO	app.log, audit.log	Scan started/completed, file uploaded, file edited, PostgreSQL INSERT completed
WARNING	app.log	getctime() used on Linux, large file detected
ERROR	error.log	Hash failure, PostgreSQL INSERT failure, subprocess error
CRITICAL	error.log	scan_folder inaccessible, disk full, unhandled exception

Log format: timestamp | level | module | message

Example audit log entry: 2026-04-06 10:54:58 | INFO | forensic_engine | Scan completed: 3 files | 1 MODIFIED | 2 UNCHANGED

11. Deployment Considerations

The current application uses Flask's development server (app.run(debug=True)). This is not suitable for production deployment.

Setting	Development (Current)	Production (Required)
Server	Flask dev server (single-threaded)	Gunicorn WSGI (multi-worker)
Debug Mode	debug=True (exposes stack traces)	debug=False
File Paths	Hardcoded in code	config.py or .env file
Secret Key	Default string	Environment variable (os.environ)
Excel Sync	Polling every 3 seconds	Already using watchdog — watcher.py runs as separate process with debounce control

Production launch command: gunicorn -w 4 -b 0.0.0.0:8000 app:app

12. Testing Strategy

12.1 Unit Testing

Unit testing validates individual functions in isolation. Key tests include:

Test Case	Method Under Test	Expected Result
Test hash of known file	calculate_hash()	Returns correct SHA256 hex string
Test hash changes after edit	calculate_hash()	Hashes differ after modification
Test NEW status on first scan	scan_directory()	All files tagged NEW
Test MODIFIED detection	scan_directory()	Edited file tagged MODIFIED
Test evidence table INSERT on first scan	insert_data()	Row inserted in PostgreSQL with correct 8 columns including risk_level
Test append mode	insert_data() called twice	PostgreSQL row count increases; previous records preserved

Test Case	Method Under Test	Expected Result
Test get_latest_snapshot()	get_current_state()	Returns rows from MAX(scan_time) query only
Test secure_filename()	validate_filename()	../../etc/passwd becomes safe string

12.2 Integration Testing

Integration testing verifies end-to-end flows across components:

- Upload file → check database: Uploaded file appears as NEW in PostgreSQL evidence table.
- Edit file → check dashboard: Edited file appears as MODIFIED in the admin dashboard.
- GET /api/summary: JSON counts match actual PostgreSQL evidence table state.
- POST /api/scan: Returns { "status": "ok" } and PostgreSQL row count increases.
- GET /api/download: Returns a valid CSV file with correct 7 column headers.

13. Future Enhancements

The system can be further improved with the following enhancements:

- JWT-based admin authentication and role-based access control (ADMIN/VIEWER roles).
- PostgreSQL row-level integrity: add a chained_hash column storing SHA256 of the previous row to detect unauthorized database tampering.
- Replace polling sync_to_excel.py with watchdog event-driven file listener.
- PostgreSQL implemented. Future: add composite index on (scan_time, status) for faster dashboard queries at scale. Add scheduled pg_dump backups for evidence archival.
- Introduce config.py / .env for configurable paths and secret keys.
- Refactor: eliminate subprocess call — invoke ForensicEngine.scan_directory() directly.
- Deploy with Gunicorn + nginx in production with debug=False.
- Add Flask-WTF CSRF protection on all POST routes.
- Add Flask-Limiter rate limiting on /api/scan and /upload endpoints.
- Extend scan engine to monitor AWS S3 or Google Cloud Storage buckets.

14. Conclusion

The Cloud Forensics Automation for Retail Breaches system is a secure and functional file evidence collection platform designed to simulate real-world retail data breach investigation scenarios.

The system uses SHA256 cryptographic hashing for tamper detection, PostgreSQL for reliable evidence storage, and a Flask-based web interface for administrative monitoring and client file management. Files are classified as NEW, UNCHANGED, MODIFIED, or DELETED on every scan. Risk levels (CRITICAL/HIGH/MEDIUM/LOW) are assigned automatically. The watchdog-based watcher.py provides real-time automatic scanning on any file system event.

Built with Python, Flask, PostgreSQL, and watchdog, the system provides real-time forensic monitoring. The architecture separates concerns across scanner.py (forensic engine), db.py (data layer), app.py (web layer), and watcher.py (event listener), enabling clean extension to cloud storage monitoring.

Overall, it provides a practical, open-source solution for forensic evidence collection in retail cybersecurity environments, meeting the requirements of IBM Final Year Project PRJN26-172.